PayApp Build Roadmap

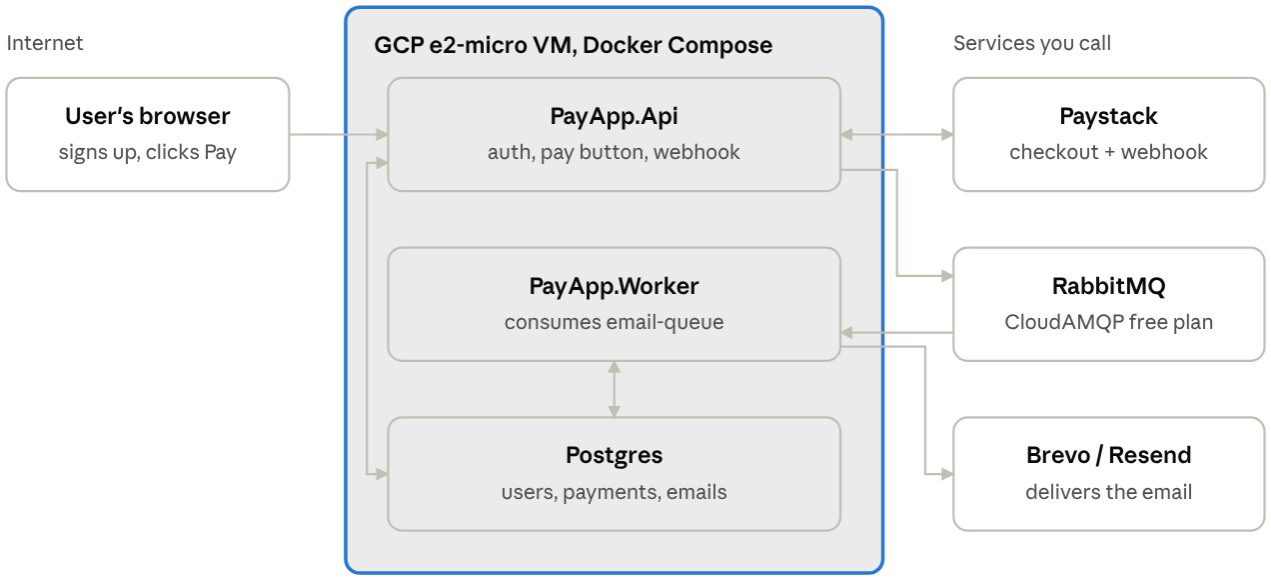
Oct 8, 2026 · @Ibukunoluwa

What we're building

A small .NET web app where users sign up, log in and click a button that pays ₦100 into your Paystack account in test mode. It sends email notifications and runs promo emails through a queue. We build it by hand first, then move parts of it to Ductape to see exactly what Ductape replaces.

Piece	Its job	Runs where
.NET API (ASP.NET Core)	Sign-up, login, the pay button, Paystack's webhook	Docker container on the VM
Email worker (.NET Worker Service)	Listens to the queue, sends each email	Docker container on the VM
Postgres	Users, payments, email records	Docker container on the VM
RabbitMQ	Holds email jobs until the worker takes them	Docker locally; CloudAMQP free plan in the cloud
Paystack (test mode)	Checkout page and payment confirmation	Paystack's servers
Mailpit / Brevo or Resend	Fake inbox locally; real email delivery in the cloud	Docker locally; provider's servers in the cloud
GCP e2-micro VM	The one rented Linux machine that hosts our three containers	Google Cloud, free tier, US region

Your code and data live on one VM; payments, queue and email are calls out



architecture · 3 containers on the VM, 3 outside services

The API publishes email jobs to RabbitMQ and the worker consumes them, so the two never call each other directly.

Tech choices: .NET 8, EF Core with the Npgsql provider, ASP.NET Core Identity for auth, Docker Compose to run everything together. The same compose setup runs on your laptop and on the VM.

Phase 0: Install your tools

Goal: your laptop can build .NET apps and run containers. Done when `dotnet --version`, `docker run hello-world` and `git --version` all work.

- [ ] Install the .NET 8 SDK
- [ ] Install Docker Desktop and confirm `docker run hello-world` works
- [ ] Install Git and create a GitHub account
- [ ] Install an editor: Visual Studio, VS Code with the C# Dev Kit, or Rider
- [ ] Install an API tester: Postman or Bruno
- [ ] Install ngrok and create a free account (gives localhost a public URL for Paystack webhooks)
- [ ] Create a GitHub repo, and add a .NET `.gitignore` before the first commit so keys never get pushed

Phase 1: Accounts and integrations

Goal: every outside service exists and you hold its test keys. Locally, everything except Paystack runs free in Docker; cloud accounts are only needed by Phase 6.

Service	What for	Local (now)	Cloud (Phase 6)
Paystack	Taking the ₦100 test payment	Test mode, works instantly, no business verification	Same test keys
Mailpit	Fake inbox to see emails during development	Docker container, inbox at localhost:8025	Not used
Brevo or Resend	Real email delivery	Not needed	Free tier, needs your own domain with SPF/DKIM records
RabbitMQ	The queue for email jobs	Docker container, dashboard at localhost:15672	CloudAMQP free plan
Postgres	The database	Docker container	Docker container on the VM
Google Cloud	Hosting	Not needed	One free e2-micro VM

- [ ] Sign up for Paystack, stay in test mode, copy the test secret key and test public key
- [ ] Save keys in .NET user-secrets (`dotnet user-secrets`), never in code or appsettings.json
- [ ] Buy a domain name (needed for real email and a proper URL later)
- [ ] Pick your email provider: Brevo (strong on promo emails) or Resend (simplest API)

Paystack amounts are in kobo, so ₦100 is `10000`. The button is a collection (money in), not a transfer (money out); transfers are what need verification.

Phase 2: Project structure and database

Goal: one solution with three projects, Postgres and RabbitMQ running in Docker, and the tables created by EF Core migrations.

```
PayApp.sln
├── PayApp.Api/      dotnet new webapi    HTTP: auth, pay button, Paystack
webhook
├── PayApp.Worker/   dotnet new worker    no HTTP: consumes the email queue
└── PayApp.Core/     dotnet new classlib  shared: entities, DbContext, email
sender
```

- [ ] Create the solution and the three projects; Api and Worker both reference Core
- [ ] Write `docker-compose.yml` with postgres, rabbitmq (management image) and mailpit
- [ ] Add EF Core with the Npgsql provider to Core
- [ ] Create the entities below, add a migration, run it, and inspect the tables with a DB client

Table	Key columns	Notes
AspNetUsers (from Identity)	Id, Email, PasswordHash + FullName, MarketingOptIn, CreatedAt	Identity creates it; you add the extra fields
payments	id uuid, user_id, reference (unique), amount_kobo bigint, currency, status, paystack_txn_id, paid_at, created_at	Status: pending, success, failed, abandoned
payment_webhook_events	id, reference, event_type, payload jsonb, received_at	Raw log so a webhook is never processed twice
email_campaigns	id, subject, body_html, status, created_at, sent_at	Status: draft, sending, sent
email_messages	id, campaign_id, user_id, to_address, kind, status, attempts, last_error, created_at, sent_at	Kind: verification, receipt, promo

MySQL to Postgres habits: use `uuid` and `timestamptz`, use `jsonb` for raw JSON, and store money as whole kobo in `bigint`, never as a float.

Phase 3: Authentication

Goal: a user can register, confirm their email, log in and log out. Use ASP.NET Core Identity; it handles password hashing and lockouts, so you never store or compare passwords yourself.

- [ ] Add Identity to the API with Postgres as its store
- [ ] Register endpoint or page: create the user, then send a verification email (straight to Mailpit for now; it moves onto the queue in Phase 5)
- [ ] Confirm-email link that marks the account verified
- [ ] Login and logout
- [ ] Protect the pay endpoint so only logged-in, verified users can use it
- [ ] Add a marketing opt-in checkbox at sign-up (feeds `MarketingOptIn`)

Test it: register in Postman or the browser, open the email in Mailpit, click the link, then log in.

Phase 4: Paystack test payment

Goal: clicking "Pay ₦100" ends with a `success` row in `payments`, confirmed by Paystack's webhook, never by the browser.

1. User clicks Pay. The API creates a `payments` row (status `pending`, a unique reference, 10000 kobo).
2. The API calls Paystack's Initialize Transaction with the email, amount and reference, and gets back a checkout URL.
3. The user is redirected to Paystack's checkout and pays with a Paystack test card.
4. Paystack calls your webhook endpoint with a `charge.success` event.
5. The API checks the webhook signature, logs the raw event, ignores it if already processed, then marks the payment `success`.
6. Optionally, it calls Verify Transaction to double-check the amount before trusting it.

- [ ] Store the Paystack test secret key in user-secrets
- [ ] Build steps 1 to 3 and confirm the checkout page opens
- [ ] Run ngrok and paste its URL into Paystack's test webhook setting
- [ ] Build the webhook endpoint with signature check and idempotency (step 5)
- [ ] Show the user their payment status after redirect

The webhook is the source of truth because a user can close the tab before your redirect page loads.

Phase 5: Email and the queue

Goal: the API never sends email itself. It writes an `email_messages` row and puts a message on RabbitMQ; the worker sends it. A promo to 500 users returns instantly and survives a crash.

- [ ] In Core, write one `IEmailSender` with a Mailpit (SMTP) version now and a Brevo/Resend version later
- [ ] In the API, add a publisher that puts `{ emailMessageId }` on an `email-queue`
- [ ] Move verification and payment-receipt emails onto the queue
- [ ] Build the worker: a `BackgroundService` registered with `AddHostedService`, consuming `email-queue` with manual acknowledgements
- [ ] On success: mark the row `sent` and acknowledge. On failure: increase `attempts`, retry, and after 3 failures send it to a dead-letter queue
- [ ] Add an admin-only "send promo" endpoint: create a campaign, then one row and one queue message per opted-in user
- [ ] Add an unsubscribe link to every promo email
- [ ] Experiment: stop the worker, send a promo, watch messages pile up in the RabbitMQ dashboard, start the worker and watch them drain

The worker is event-driven, not a cron job: RabbitMQ pushes each message to it the moment it arrives.

Phase 6: Deploy to Google Cloud

Goal: the same Docker Compose setup runs on one free e2-micro VM, reachable at your domain over HTTPS, for close to zero cost. GCP has no hard spending cap; budgets only send alerts, so the guardrails come first.

Cost guardrails (before creating anything):

- [ ] Check Billing: free trial (cannot be charged until you upgrade) or paid account
- [ ] Create a budget of $5 with alerts at 50%, 90% and 100%
- [ ] Use only `us-central1`, `us-east1` or `us-west1`
- [ ] Do not create Cloud SQL; it is not in the free tier
- [ ] Never commit keys or service-account files; check `.gitignore`

Deploy:

- [ ] Create the e2-micro VM (Ubuntu), install Docker, add swap space (the VM has about 1 GB of RAM)
- [ ] Create a CloudAMQP free instance and point the app at it
- [ ] Set up your email provider with your domain's SPF and DKIM records
- [ ] Copy the compose file to the VM; put secrets in a `.env` file on the VM only
- [ ] Firewall: open only ports 80 and 443; Postgres stays private

- [ ] Put a reverse proxy (Caddy) in front for automatic HTTPS, and point your domain at the VM
- [ ] Change Paystack's webhook URL from ngrok to your domain
- [ ] Add a nightly cron job that dumps the database and stores the backup off the VM

Phase 7: Move pieces to Ductape

Goal: replace one piece at a time with Ductape and write down what got simpler and what you gave up. Ductape connects to your infrastructure; it does not host your database.

Order	What moves	Hand-built version it replaces
1	Paystack as a Ductape app, called by name	Your HTTP client, auth header and retry code
2	Email as a Ductape notification	Your `IEmailSender` and provider config
3	RabbitMQ as a Ductape broker	Your publisher and connection setup
4	Postgres as a Ductape database resource	Your connection strings per environment
5	Sign-up and payment as Ductape Features (steps with rollback)	The orchestration code in your endpoints

- [ ] Create a Ductape product with `dev` and `prd` environments
- [ ] Install the CLI (`npm install -g @ductape/cli` , then `ductape login`); this needs Node.js
- [ ] Add `Ductape.Sdk` (NuGet, .NET 8, still pre-1.0) to the API
- [ ] Use a restricted database user for Ductape, never the admin account
- [ ] Before connecting the database, ask Ductape whether it must be reachable from their servers or only from your app

Ductape is in beta and Features are marked preview, so expect API changes and check method names against the package.

Concepts

Term	Plain meaning
VM	A rented computer in a data centre that stays on with a public address
Container	One app packaged with everything it needs, run by Docker
Docker Compose	One file that starts all your containers together and networks them
Queue	A waiting line of jobs; producers add, consumers take, nothing is lost if a consumer is down
Producer / consumer	The API adds messages (producer); the worker takes them (consumer)
Acknowledgement	The worker telling RabbitMQ "done, delete it"; no ack means the message is retried
Dead-letter queue	Where messages go after failing too many times, for you to inspect
Background service	Code that starts with the app and keeps running, reacting to events
Cron job	A program that runs on a schedule, then exits
Webhook	A provider calling your URL to tell you something happened
Idempotency	Handling the same event twice without doing the work twice
Microservices	Separate apps with separate databases per business area; not what we are building

Sources: Ductape docs · Ductape.Sdk on NuGet · Paystack test mode · Google Cloud free program